

4-3장

2101080 정진용

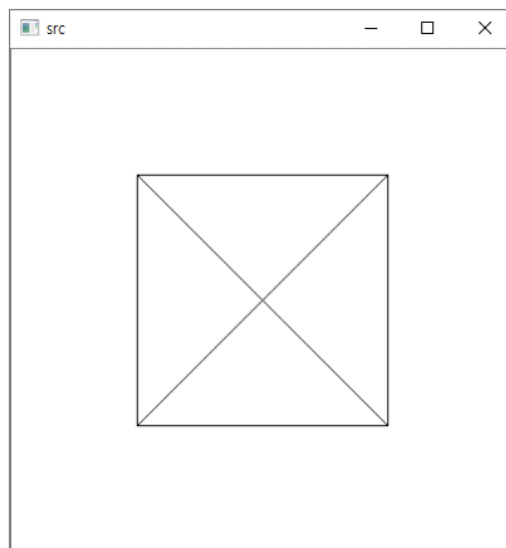
1.

(ANSWER)

실습과제1



- 400x400 영상을 생성하고 다음처럼 line 함수를 이용하여 직선을 그리시오.



코드:

```
#include <opencv2/opencv.hpp> // OpenCV 라이브러리의 기본 헤더 파일 포함
#include <iostream> // 입출력 스트림을 위한 헤더 파일 포함

using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main()
{
    // 1채널 (그레이스케일), 400x400 픽셀 크기의 흰색 이미지 생성 (픽셀값 255)
    Mat image(400, 400, CV_8UC1, Scalar(255));

    // 이미지의 크기 정보를 가져와서 imageSize에 저장 (width, height 포함)
    Size imageSize = image.size();

    // 정사각형 4등분을 위한 기준 좌표 계산 (height의 1/4)
    int a = imageSize.height / 4;

    // 정사각형 4등분 기준점 설정:
    // 왼쪽 위, 오른쪽 위, 왼쪽 아래, 오른쪽 아래의 4개 지점 정의
    vector<Size> xy = {
        {Size(a, a)}, // 왼쪽 위 지점 (100, 100)
        {Size(a * 3, a)}, // 오른쪽 위 지점 (300, 100)
        {Size(a, a * 3)}, // 왼쪽 아래 지점 (100, 300)
        {Size(a * 3, a * 3)} // 오른쪽 아래 지점 (300, 300)
    };

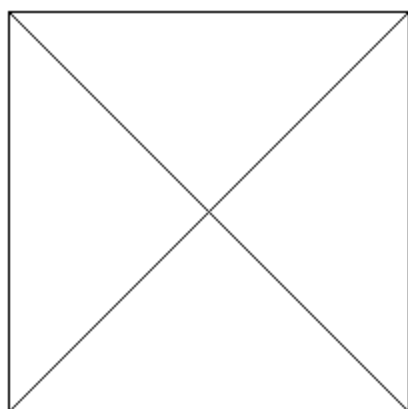
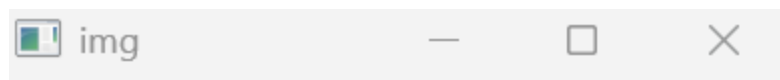
    // 선 그리기
    line(image, xy[0], xy[1], Scalar(0));
    line(image, xy[1], xy[3], Scalar(0));
    line(image, xy[2], xy[3], Scalar(0));
    line(image, xy[0], xy[2], Scalar(0));
    line(image, xy[0], xy[3], Scalar(0));
    line(image, xy[1], xy[2], Scalar(0));

    // 윈도우에 이미지 출력
    imshow("img", image);

    // 키 입력이 있을 때까지 대기
    waitKey();

    // 프로그램 종료
    return 0;
}
```

콘솔:



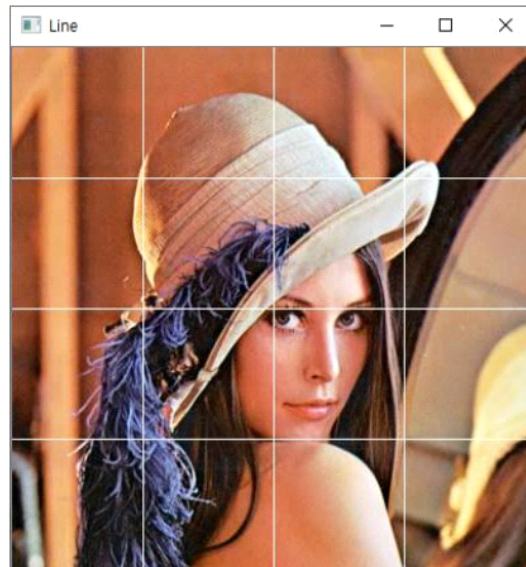
2.

(ANSWER)

실습과제2



- Lenna 영상에 다음 실행결과처럼 화면을 4등분하는 가로, 세로선을 그리시오. 반드시 line함수를 이용할 것. 라인 스타일 수정 가능함.



코드:

```
#include <opencv2/opencv.hpp> // OpenCV의 핵심 기능을 포함한 헤더 파일
#include <iostream>             // 입출력을 위한 표준 C++ 라이브러리

using namespace cv;            // OpenCV 네임스페이스 사용
using namespace std;          // C++ 표준 네임스페이스 사용

int main()
{
    Mat image = imread("Lenna.bmp", IMREAD_COLOR); // 컬러 이미지로 Lenna.bmp 파일 불러오기

    if (image.empty()) { // 이미지가 제대로 로드되지 않았을 경우
        cerr << "이미지를 불러올 수 없습니다." << endl; // 에러 메시지 출력
        return -1; // 프로그램 비정상 종료
    }

    Size imagesize = image.size(); // 이미지의 가로(width), 세로(height) 크기 얻기

    int imageheight = imagesize.height / 4; // 세로 방향을 4등분한 높이 계산
    int imagewidth = imagesize.width / 4; // 가로 방향을 4등분한 너비 계산

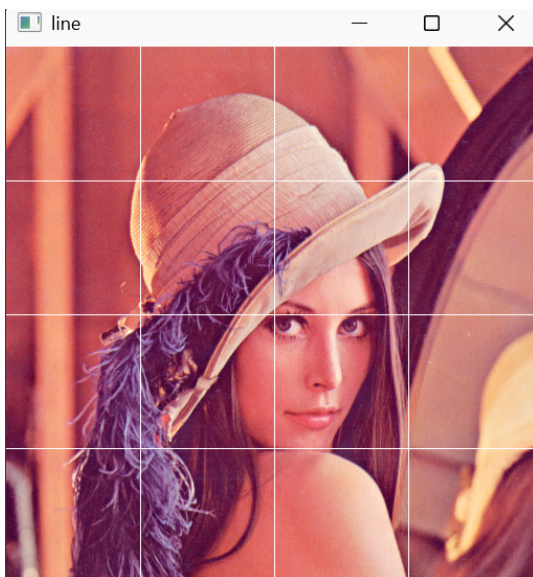
    for (int i = 1; i < 4; i++) { // 1/4, 2/4, 3/4 위치에 선을 그리기 위한 반복문
        line(image,
            Point(imagewidth * i, 0), // 세로선 시작점 (i*너비, 0)
            Point(imagewidth * i, imageheight * 4), // 세로선 끝점 (i*너비, 전체 높이)
            Scalar(255, 255, 255)); // 선 색상: 흰색

        line(image,
            Point(0, imageheight * i), // 가로선 시작점 (0, i*높이)
            Point(imagesize.width, imageheight * i), // 가로선 끝점 (전체 너비, i*높이)
            Scalar(255, 255, 255)); // 선 색상: 흰색
    }

    imshow("line", image); // 'line'이라는 창 이름으로 이미지 출력
    waitKey(); // 키 입력이 있을 때까지 대기

    return 0; // 프로그램 정상 종료
}
```

콘솔:

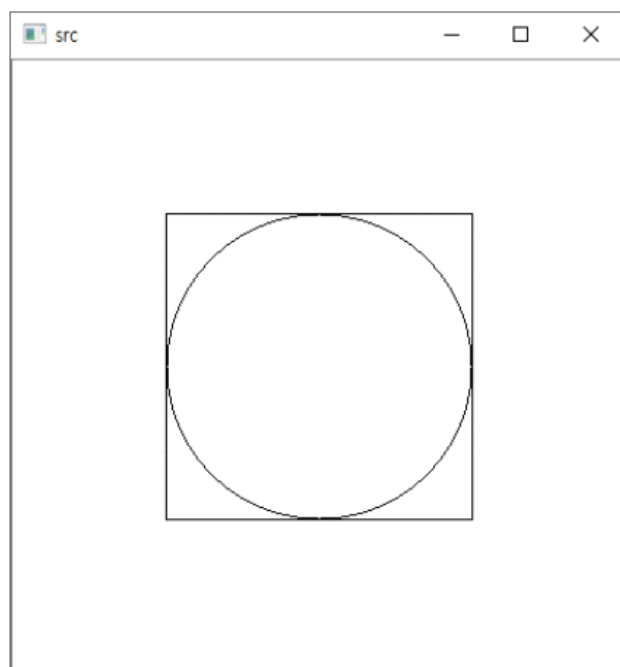


3.

(ANSWER)

실습과제3

- 400x400 영상을 생성하고 다음처럼 도형을 그리시오.



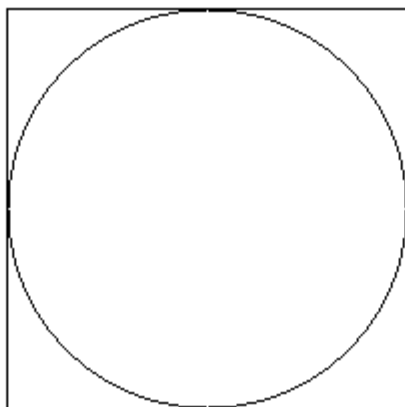
코드:

```
#include <opencv2/opencv.hpp> // OpenCV 함수들을 사용하기 위한 헤더 파일
#include <iostream>           // 표준 입출력을 위한 헤더 파일

using namespace cv;          // OpenCV 네임스페이스 사용
using namespace std;         // C++ 표준 네임스페이스 사용

int main()
{
    Mat image(400, 400, CV_8UC1, Scalar(255)); // 크기 400x400, 단일 채널(그레이스케일), 초기값 흰색(255)인 이미지 생성
    Size imagesize = image.size(); // 이미지의 가로, 세로 크기 정보를 imagesize에 저장
    rectangle(image, Point(100, 100), Point(300, 300), Scalar(0)); // (100,100)에서 (300,300)까지 검은색 사각형 그리기
    circle(image, Point(imagesize.width / 2, imagesize.height / 2), 100, Scalar(0)); // 이미지 중심에 반지름 100의 검은색 원 그리기
    imshow("image", image); // 'image'라는 창 이름으로 이미지 출력
    waitKey(); // 키 입력이 있을 때까지 대기
    return 0; // 프로그램 정상 종료
}
```

콘솔:



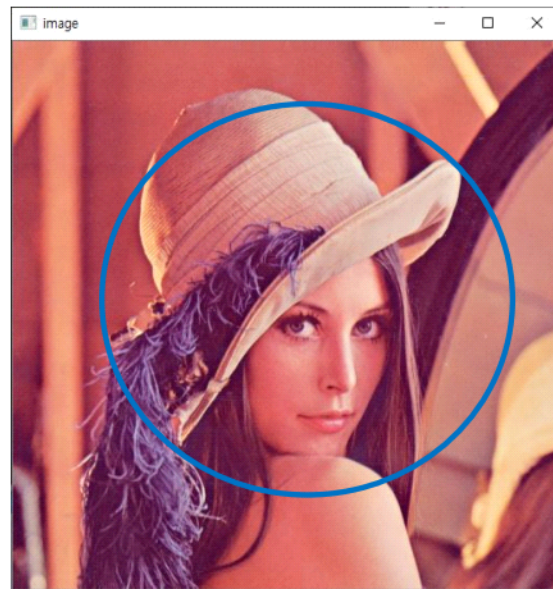
4.

(ANSWER)

실습과제4



- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 다음 실행결과처럼 얼굴주위에 파란색 원을 그리시오. 필요한 좌표값은 그림판 프로그램을 이용하여 구하시오.



코드:

```
#include <opencv2/opencv.hpp> // OpenCV 라이브러리의 핵심 함수들을 사용하기 위한 헤더 파일
#include <iostream> // 표준 입출력 스트림을 위한 헤더 파일

using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main()
{
    Mat image = imread("lenna.bmp"); // lenna.bmp 이미지를 컬러 (BGR)로 불러와 Mat 객체 image에 저장

    if (image.empty()) { // 이미지 파일이 제대로 불러와지지 않았는지 확인
        cerr << "이미지를 불러올 수 없습니다." << endl; // 오류 메시지 출력
        return -1; // 비정상 종료 코드 반환
    }

    Size imagesize = image.size(); // 이미지의 가로, 세로 크기를 imagesize에 저장

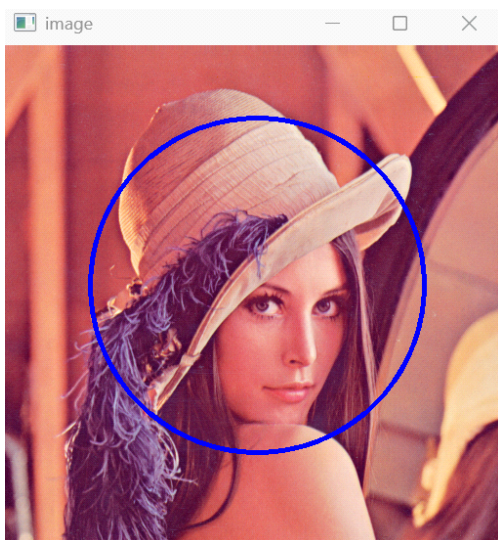
    circle(image,
        Point(260, 245), // 원의 중심 좌표 설정 (x = 260, y = 245)
        171, // 원의 반지름 설정
        Scalar(255, 0, 0), // 원의 색상 설정 (파란색, BGR 기준)
        3); // 원의 테두리 두께 설정 (3픽셀)

    imshow("image", image); // 'image'라는 이름의 창에 이미지 출력

    waitKey(); // 키 입력이 있을 때까지 대기

    return 0; // 프로그램 정상 종료
}
```

콘솔:



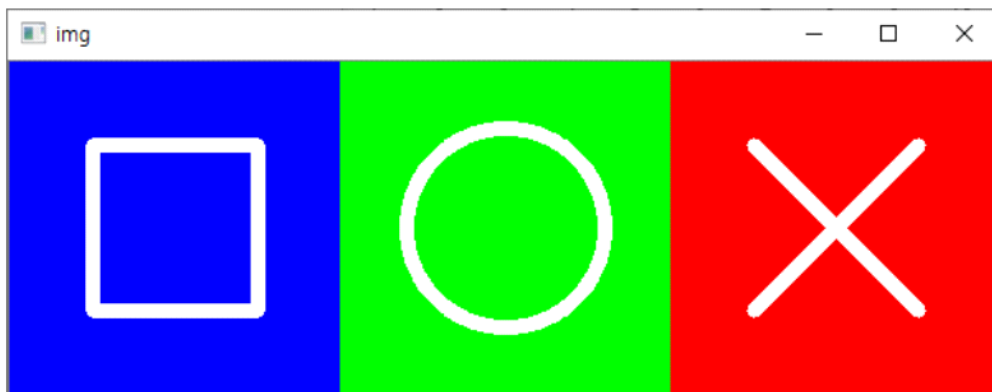
5.

(ANSWER)

실습과제5



- 600X200 컬러영상을 만들고 다음 실행결과처럼 Red, Green, Blue로 배경을 만들고 각 도형을 그리시오. 힌트: 3장 부분행렬(관심영역)을 응용해볼 것, 3개의 관심영역을 만들어 처리하면 좌표 계산이 쉬움



코드:

```
#include <opencv2/opencv.hpp> // OpenCV 라이브러리의 핵심 기능 포함
#include <iostream> // C++ 표준 입출력 스트림 헤더

using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

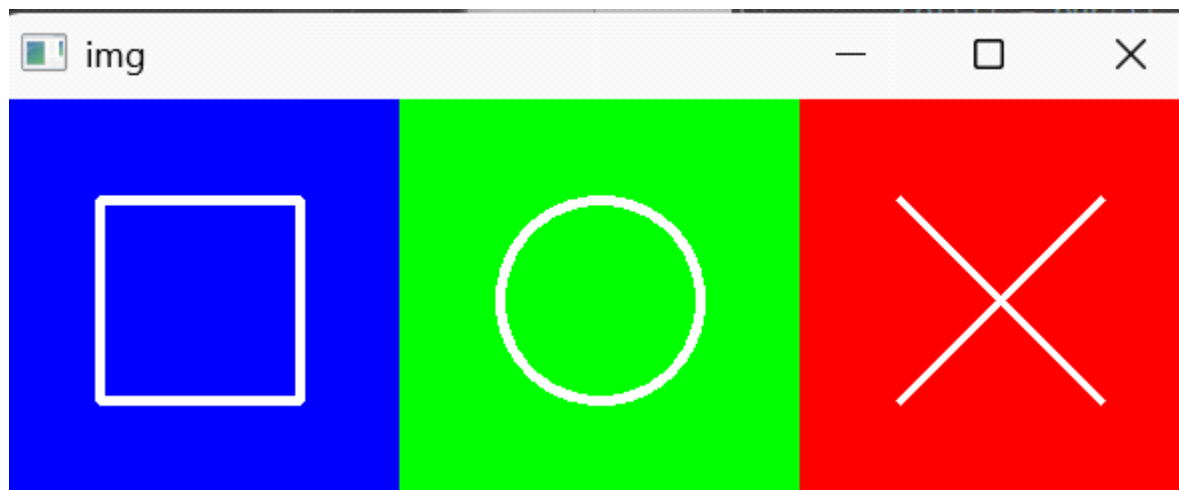
int main()
{
    Mat image(200, 600, CV_8UC3); // 세로 200, 가로 600, 3채널(BGR) 컬러 이미지 생성 (초기값 없음 → 쓰레기 값 있음)
    Mat roi[3]; // 영역별 (Region of Interest) 이미지를 저장할 Mat 배열 선언
    Scalar bgr[] = { // BGR 색상 배열 정의: 파랑, 초록, 빨강, 흰색
        Scalar(255, 0, 0), // 파란색
        Scalar(0, 255, 0), // 초록색
        Scalar(0, 0, 255), // 빨간색
        Scalar(255, 255, 255) // 흰색
    };

    for (int i = 0; i < 3; i++) {
        roi[i] = image(Rect(200 * i, 0, 200, 200)); // 이미지에서 가로로 200씩 나눈 각 영역 추출
        roi[i] = bgr[i]; // 각 영역을 bgr 배열의 색으로 채움
    }

    rectangle(roi[0], Point(50, 50), Point(150, 150), bgr[3], 4); // 첫 번째 영역에 흰색 테두리 사각형 그리기 (두께 4)
    circle(roi[1], Point(100, 100), 50, bgr[3], 4); // 두 번째 영역에 흰색 테두리 원 그리기 (중심 100,100 / 반지름 50 / 두께 4)
    line(roi[2], Point(50, 50), Point(150, 150), bgr[3], 4); // 세 번째 영역에 흰색 대각선 긋기 (왼쪽 위 → 오른쪽 아래, 두께 4)
    line(roi[2], Point(150, 50), Point(50, 150), bgr[3], 4); // 세 번째 영역에 흰색 대각선 긋기 (오른쪽 위 → 왼쪽 아래, 두께 4)

    imshow("img", image); // 'img'라는 이름의 창에 최종 이미지 출력
    waitKey(); // 키 입력이 있을 때까지 대기
}
```

콘솔:



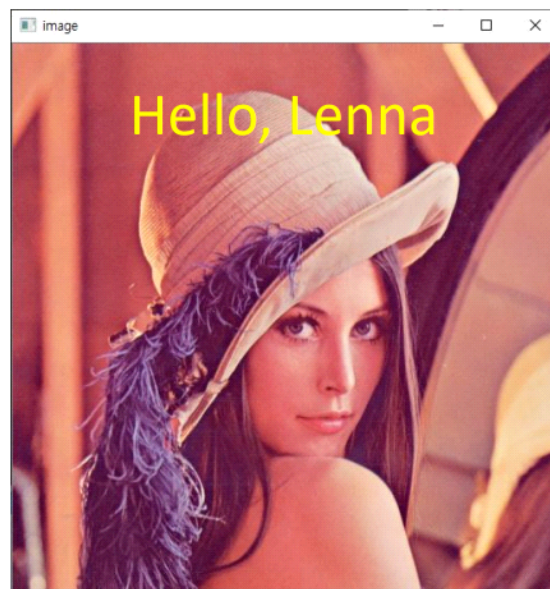
6.

(ANSWER)

실습과제6



- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 다음 실행결과처럼 문자열을 출력하시오. 폰트설정은 원하는 대로 하시오.
- 문자열이 상단의 중앙에 위치하도록 위치를 조정하시오.



코드:

```
#include <opencv2/opencv.hpp> // OpenCV의 기본 기능을 포함하는 헤더 파일
#include <iostream> // 표준 입출력 스트림 사용을 위한 헤더

using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main() {
    // Lenna 이미지 읽기 (컬러 모드)
    Mat image = imread("lenna.bmp", IMREAD_COLOR); // 컬러 이미지로 "lenna.bmp" 파일 읽기

    if (image.empty()) { // 이미지 파일이 제대로 로드되지 않았을 경우
        cerr << "이미지를 불러올 수 없습니다." << endl; // 에러 메시지 출력
        return -1; // 프로그램 비정상 종료
    }

    // 출력할 텍스트와 폰트 설정
    string text = "Hello, Lenna"; // 출력할 문자열 지정
    int fontFace = FONT_HERSHEY_SIMPLEX; // 사용할 글꼴 스타일 설정
    double fontScale = 1.5; // 글꼴 크기 설정
    int thickness = 7; // 글꼴 두께 설정
    Scalar fontColor = Scalar(0, 255, 255); // 글자 색상 (노란색, BGR 기준)

    Size textSize = getTextSize(text, fontFace, fontScale, thickness, 0); // 텍스트 크기 계산

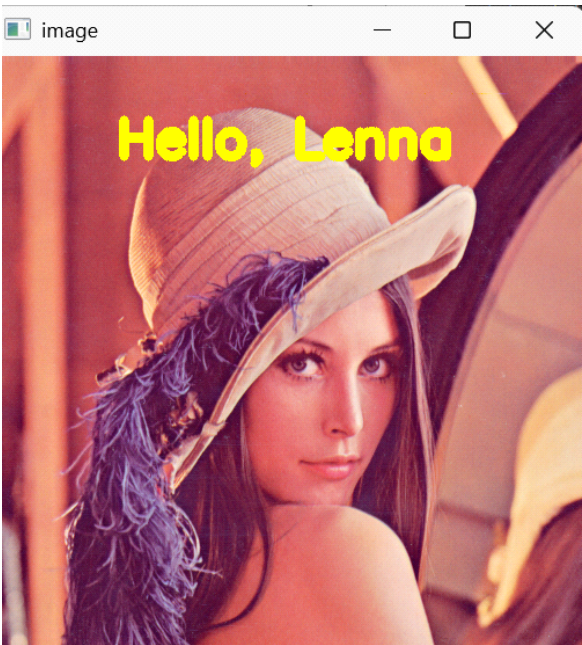
    Point textOrg((image.cols - textSize.width) / 2, textSize.height + 50); // 이미지 중앙 상단에 텍스트 시작 위치 설정

    putText(image, text, textOrg, fontFace, fontScale, fontColor, thickness); // 이미지 위에 텍스트 출력

    imshow("image", image); // "image"라는 이름의 윈도우에 이미지 출력
    waitKey(0); // 키보드 입력이 있을 때까지 대기

    return 0; // 프로그램 정상 종료
}
```

콘솔:



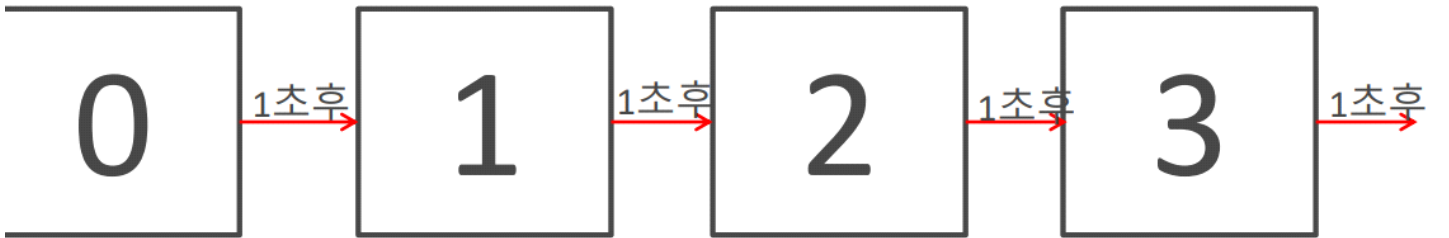
7.

(ANSWER)

실습과제7



- 다음처럼 출력하는 초시계를 만들어 보시오.
- 윈도우 사이즈는 300x300으로 만드시오.
- 초기상태는 0을 출력하고 대기한다. 's'키를 누르면 카운트를 시작하고 't'키를 누르면 정지한다. 'r'키를 누르면 0으로 리셋 된다. 'q'키를 입력하면 프로그램을 종료한다.
- 시간제어 및 문자입력을 위하여 waitKey함수를 사용하라.
- 숫자출력을 위하여 변수의 값을 문자열로 변환할 때 format함수를 사용하고 문자열을 출력할 때 putText함수를 사용할 것
- 반복문은 1번만 사용하라. 같은 함수를 2번 이상 호출하지 말 것



코드:

```
#include <opencv2/opencv.hpp> // OpenCV 함수 사용을 위한 헤더 파일
#include <iostream> // 입출력 스트림 사용을 위한 헤더 파일
using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main() {
    // 흰색 배경의 300x300 크기 흑백 이미지 생성
    Mat image(300, 300, CV_8UC1, Scalar(255)); // 크기 300x300, 8비트 1채널(흑백), 흰색으로 초기화

    bool a = false; // 타이머 동작 여부를 저장하는 변수
    int count = 0; // 카운트 값 (초 단위)
    int fontface = FONT_HERSHEY_SIMPLEX; // 사용할 글꼴 스타일 설정
    double fontscale = 8; // 글꼴 크기 설정
    int thickness = 2; // 글꼴 두께 설정
    Scalar fontcolor = Scalar(0, 0, 0); // 글자 색상 (검정)

    int key = 0; // 키 입력 값을 저장할 변수

    imshow("시간초", image); // 초기 이미지 출력

    while (true) { // 무한 반복 루프 시작
        image = Scalar(255); // 이미지를 흰색으로 초기화

        string b = format("%d", count); // 현재 카운트를 문자열로 변환
        Size textsize = getTextSize(b, fontface, fontscale, thickness, 0); // 텍스트 크기 계산

        Point c(((image.cols - textsize.width) / 2), ((image.rows + textsize.height) / 2)); // 중앙에 텍스트 위치 설정

        putText(image, b, c, fontface, fontscale, fontcolor, thickness); // 이미지에 텍스트 출력

        imshow("시간초", image); // 이미지 출력

        key = waitKey(1000); // 1초 대기하며 키 입력 받기

        if (key == 's') { // 's' 키를 누르면 타이머 시작
            a = true;
        }
        else if (key == 't') { // 't' 키를 누르면 타이머 일시정지
            a = false;
        }
        else if (key == 'r') { // 'r' 키를 누르면 타이머 리셋
            count = 0;
        }
        else if (key == 'q') { // 'q' 키를 누르면 프로그램 종료
            cout << "프로그램 종료";
            break;
        }

        if (a) count++; // 타이머가 작동 중이면 1초 증가
    }

    return 0; // 프로그램 종료
```

콘솔:동영상 파일로 제출하겠습니다

8.

(ANSWER)

실습과제8

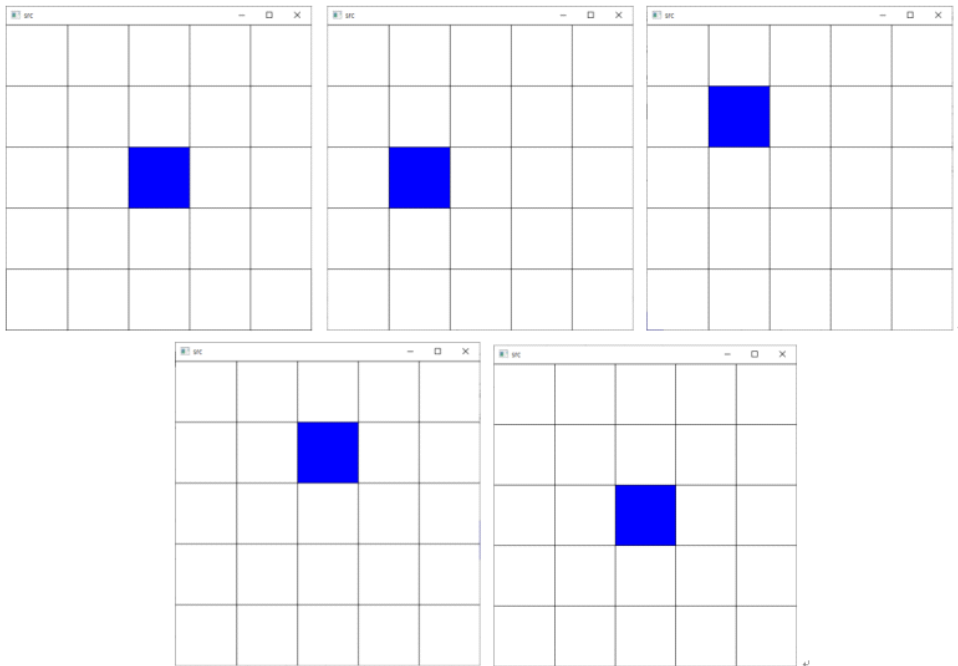


- 키보드를 이용하여 파란색 버튼을 움직이는 프로그램을 만들고자 한다. 아래 지시대로 작성 하시오.
- ① 초기 화면은 폭500, 높이500픽셀의 3채널 컬러영상으로 설정하라. 첫 번째 그림처럼 바둑판모양으로 직선을 출력하고 파란색버튼이 가운데 위치하도록 하라.
- ② 두번째 그림처럼 'l' 키를 누르면 현재위치에서 왼쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ③ 세번째 그림처럼 'u' 키를 누르면 현재위치에서 위쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ④ 네번째 그림처럼 'r' 키를 누르면 현재위치에서 오른쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ⑤ 다섯번째 그림처럼 'd' 키를 누르면 현재위치에서 아래쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ⑥ 'q'키를 누르면 종료하도록 하라.

실습과제8



- waitKey함수로 키보드 입력을 받고 새로운 버튼의 좌표를 계산하라. 버튼의 이전 위치도 항상 저장하여 이전 버튼은 지우고 새버튼을 그린다.
- 버튼의 좌표가 윈도우를 벗어난 경우 현재 값을 유지하라.



코드:

```
// 8번
#include <opencv2/opencv.hpp> // OpenCV 라이브러리 포함
#include <iostream> // 표준 입출력 스트림 사용
using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // C++ 표준 네임스페이스 사용

int main() {
    Mat image(500, 500, CV_8UC3, Scalar(255, 255, 255)); // 흰색 배경의 500x500 크기 컬러 이미지 생성
    Size imagesize = image.size(); // 이미지의 크기 정보 저장
    int x = 200, y = 200, xx = -1, yy = -1; // 초기 사각형 위치 설정, xx/yy는 새 위치 후보
    Rect a(x, y, 100, 100); // 초기 사각형 생성 (100x100 크기)

    for (int i = 1; i < 5; i++) { // 격자 무늬 그리기 (수직, 수평 100픽셀 간격)
        line(image, Point(100 + i, 0), Point(100 + i, imagesize.height), Scalar(0, 0, 0)); // 수직선
        line(image, Point(0, 100 + i), Point(imagesize.width, 100 + i), Scalar(0, 0, 0)); // 수평선
    }

    rectangle(image, a, Scalar(255, 0, 0), -1); // 파란색 사각형을 채워서 그리기

    while (true) {
        image = Scalar(255, 255, 255); // 이미지 초기화 (다시 흰색 배경으로)

        for (int i = 1; i < 5; i++) { // 격자 다시 그리기
            line(image, Point(100 + i, 0), Point(100 + i, imagesize.height), Scalar(0, 0, 0)); // 수직선
            line(image, Point(0, 100 + i), Point(imagesize.width, 100 + i), Scalar(0, 0, 0)); // 수평선
        }

        if ((0 <= xx && xx <= 400) && (0 <= yy && yy <= 400)) { // 경계 체크: 사각형이 벗어나지 않도록
            x = xx; // 새로운 위치가 유효하면 x에 적용
            y = yy;
        }
        else {
            xx = x; // 무효한 위치일 경우, 기존 위치로 복구
            yy = y;
        }

        Rect a(x, y, 100, 100); // 새로운 위치에서 사각형 생성
        rectangle(image, a, Scalar(255, 0, 0), -1); // 파란색 사각형 채워서 그리기
        imshow("src", image); // 이미지 화면에 출력
        int key = waitKey(); // 키 입력 대기

        if (key == 'l') { // 'l' 입력 시 왼쪽으로 이동
            xx = x - 100;
        }
        else if (key == 'r') { // 'r' 입력 시 오른쪽으로 이동
            xx = x + 100;
        }
        else if (key == 'u') { // 'u' 입력 시 위로 이동
            yy = y - 100;
        }
        else if (key == 'd') { // 'd' 입력 시 아래로 이동
            yy = y + 100;
        }
        else if (key == 27) break; // ESC 키 누르면 종료
    }
}
```

콘솔: 동영상 파일로 제출하겠습니다

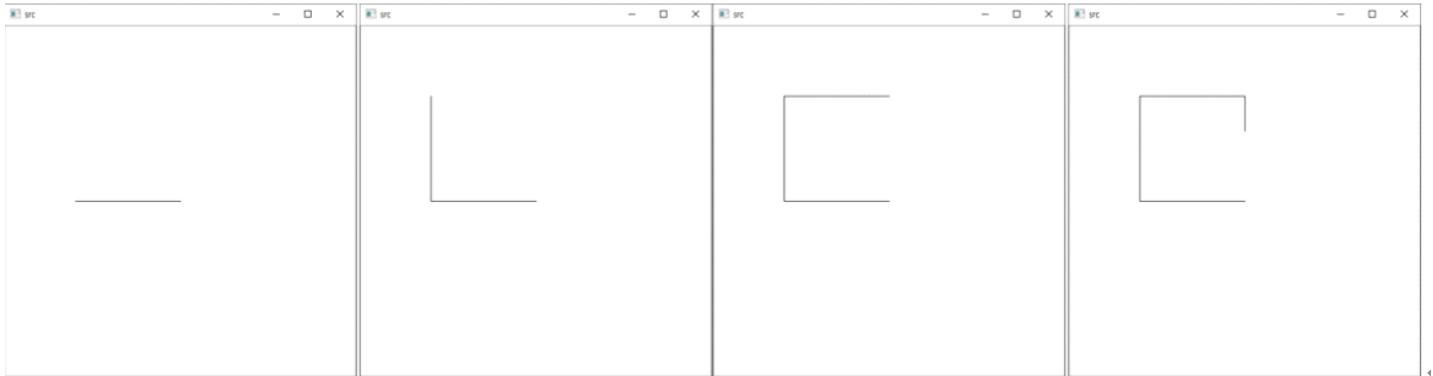
9.

(ANSWER)

실습과제9



- 아래 실행 결과는 l키를 3번 반복, u키를 3번 반복, r키를 3번 반복, d키를 1번 누른 결과이다.



코드:

```
#include <opencv2/opencv.hpp> // OpenCV 라이브러리 포함
#include <iostream> // 표준 입출력 사용
using namespace cv; // OpenCV 네임스페이스 사용
using namespace std; // 표준 C++ 네임스페이스 사용

int main() {
    // (1) 500x500 크기의 컬러 영상 생성, 흰색 배경으로 초기화
    Mat image(500, 500, CV_8UC3, Scalar(255, 255, 255)); // 이미지 생성 (흰색 배경)

    // 시작 위치는 화면 중심 (250, 250)
    Point current(250, 250);
    Point next = current;

    // 이미지 출력용 창 생성
    imshow("Draw Lines", image);

    while (true) {
        int key = waitKey();

        // 'q' 키가 눌리면 루프 종료
        if (key == 'q') break;

        // 다음 좌표를 현재 위치로 초기화
        next = current;

        // 키에 따라 이동 방향 결정
        if (key == 'l') { // 'l' 키: 왼쪽으로 50픽셀
            next.x -= 50;
        }
        else if (key == 'u') { // 'u' 키: 위로 50픽셀
            next.y -= 50;
        }
        else if (key == 'r') { // 'r' 키: 오른쪽으로 50픽셀
            next.x += 50;
        }
        else if (key == 'd') { // 'd' 키: 아래로 50픽셀
            next.y += 50;
        }
        else {
            // 유효하지 않은 키는 무시하고 다음 반복으로 (continue 없이 처리)
        }

        // 이동 후 좌표가 이미지 범위 내에 있을 경우만 선을 그림
        if (next.x >= 0 && next.x <= 500 && next.y >= 0 && next.y <= 500) {
            line(image, current, next, Scalar(0, 0, 0), 2); // 선 그리기 (검정색, 두께 2)
            current = next;
        }

        imshow("Draw Lines", image);
    }

    return 0;
}
```

콘솔: 동영상 파일로 제출하겠습니다.